

Network Slicing with Ryu and Mininet

Progetto d'esame di Networks and Cloud Infrastructures

Studente: Pasquale Cerottino

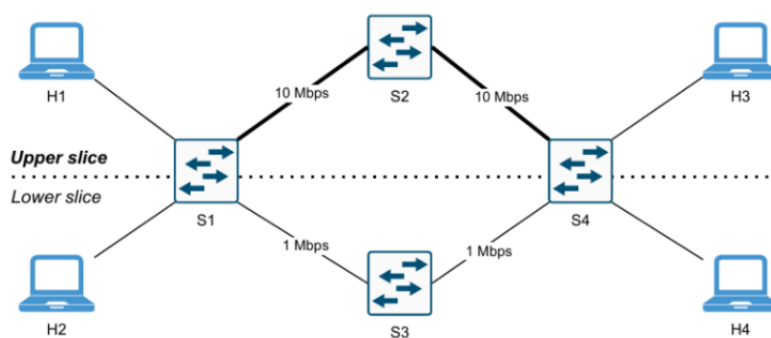
Matricola: DE9000177

1. Il problema

Il progetto implementa il network slicing in una rete SDN emulata con *Mininet* e controllata da Ryu tramite *OpenFlow* 1.3. Il network slicing suddivide un'unica infrastruttura fisica in più reti logiche (slice), ciascuna con un proprio percorso e proprie risorse, isolate tra loro. In riferimento alla topologia riportata sotto, sono state realizzate due forme di slicing:

1. Nel **topology slicing** la slice dipende da chi comunica: H1 comunica solo con H3 attraverso la slice superiore, H2 solo con H4 attraverso la slice inferiore.
2. Nel **service slicing** la slice dipende dal tipo di traffico: il traffico video, identificato come UDP con porta di destinazione 9999, usa la slice a banda maggiore, mentre il resto del traffico usa l'altra.

Entrambe le soluzioni condividono la stessa topologia e la stessa impostazione del controller: configurazione reattiva e distinzione tra switch di bordo e di transito. Cambia il criterio con cui il controller sceglie la slice di un pacchetto.



2. Creazione della topologia

La rete è descritta dallo script *topology.py*, eseguito da *Mininet* con il *Python* di sistema. Il controller è un'applicazione *Ryu* separata, eseguita in un ambiente virtuale *Python* 3.9, perché *Ryu* non è compatibile con le versioni più recenti. I due processi comunicano solo tramite *OpenFlow* sulla porta *TCP* 6653, quindi possono usare interpreti diversi.

La topologia comprende quattro host e quattro switch *Open vSwitch*, con due ruoli:

1. **Switch di bordo (S1, S4):** collegano gli host alla rete. A S1 sono collegati H1 e H2, a S4 sono collegati H3 e H4.
2. **Switch di transito (S2, S3):** realizzano i due percorsi alternativi tra S1 e S4.

I due percorsi costituiscono le slice:

- **Slice superiore:** S1 – S2 – S4, link a 10 *Mbit/s* con 15 *ms* di ritardo.
- **Slice inferiore:** S1 – S3 – S4, link a 1 *Mbit/s* con 30 *ms* di ritardo.

I parametri dei link sono raccolti in profili: i link tra switch hanno banda e ritardo, i link di accesso degli host solo il ritardo. Perché banda e ritardo vengano applicati, la rete è creata con *TCLink*. Senza di esso *Mininet* ignora questi parametri senza segnalarlo.

I numeri di porta sono assegnati esplicitamente, perché le regole del controller vi fanno riferimento:

- **S1:** porta 1 → H1, porta 2 → H2, porta 3 → S2, porta 4 → S3.
- **S2:** porta 1 → S1, porta 2 → S4.
- **S3:** porta 1 → S1, porta 2 → S4.
- **S4:** porta 1 → S2, porta 2 → S3, porta 3 → H3, porta 4 → H4.

Gli host hanno MAC e IP fissi: l'host *H_n* ha MAC 00: 00: 00: 00: 00: 0*n* e IP 10.0.0.*n*

La figura che segue riporta il codice prodotto per l'implementazione della topologia custom:

```
class CustomTopology(Topo):

    def create_switch(self, switch_index: int) -> str:
        switch_name = f's{switch_index}'
        return self.addSwitch(switch_name, cls=OVSKernelSwitch, protocols=PROTOCOL)

    def create_host(self, host_index: int) -> str:
        host_name = f'h{host_index}'
        mac_address = f'00:00:00:00:00:0{host_index}'
        ip_address = f'10.0.0.{host_index}'
        return self.addHost(host_name, mac=mac_address, ip=ip_address)

    def create_link(self, center, edges:list):
        for edge in edges:
            self.addLink(
                center, edge['device'], port1=edges.index(edge) + 1,
                port2=edge['port'], **edge['profile']
            )

    def build(self):
        # Build Switch
        s1, s2, s3, s4 = [self.create_switch(switch_index + 1) for switch_index in range(NUM_SWITCH)]

        # Build Host
        h1, h2, h3, h4 = [self.create_host(host_index + 1) for host_index in range(NUM_HOST)]

        # Build s1 Link
        s1_edges = [
            {'device': h1, 'port':1, 'profile':LINK_PROFILE['upper_access']},
            {'device': h2, 'port':1, 'profile':LINK_PROFILE['lower_access']},
            {'device': s2, 'port':1, 'profile':LINK_PROFILE['upper']},
            {'device': s3, 'port':1, 'profile':LINK_PROFILE['lower']},
        ]
        self.create_link(s1, s1_edges)

        # Build s4 Link
        s4_edges = [
            {'device': s2, 'port':2, 'profile':LINK_PROFILE['upper']},
            {'device': s3, 'port':2, 'profile':LINK_PROFILE['lower']},
            {'device': h3, 'port':1, 'profile':LINK_PROFILE['upper_access']},
            {'device': h4, 'port':1, 'profile':LINK_PROFILE['lower_access']},
        ]
        self.create_link(s4, s4_edges)
```

3. Topology Slicing

Il topology slicing deve isolare due coppie di host: H1 comunica solo con H3 attraverso la slice superiore, mentre H2 comunica solo con H4 attraverso la slice inferiore. L'isolamento vale anche per il broadcast, che deve restare nella slice dell'host che lo ha generato.

Progettazione

Il controller, implementato in *topology_slicing.py*, è basato su due scelte progettuali:

1. È **reattivo**. Il primo pacchetto di ogni flusso risale al controller con un messaggio *PacketIn*: il controller decide come instradarlo, installa la regola corrispondente nello switch e reinvia il pacchetto con un *PacketOut*. Da quel momento i pacchetti dello stesso flusso vengono gestiti direttamente dallo switch.
2. Le **regole** sono **per flusso**: ciascuna fa match sulla porta di ingresso e sulla coppia di MAC sorgente e destinazione; quindi, ogni switch ne riceve una per ogni flusso che lo attraversa.

Alla connessione di uno switch il controller installa soltanto due regole:

1. **Table-miss**, con match vuoto e priorità 0, che inoltra al controller ogni pacchetto privo di regola: è il meccanismo su cui si basa la configurazione reattiva.
2. **Scarto del traffico con IPv6**, con priorità 5. Le interfacce lo generano automaticamente (Neighbor Discovery, MLD) e, poiché il progetto usa solo IPv4, inoltrarlo al controller produrrebbe solo *PacketIn* superflui.

La logica del controller si basa su tre strutture dati:

1. **Tabella di controllo di accesso**: Indica per ogni host l'unico host con cui può comunicare: il traffico tra coppie non ammesse viene scartato.
2. **Tabella di inoltro**, che associa a ogni porta di ingresso la porta di uscita della slice. Sugli switch di bordo ogni porta host è legata all'uplink della propria slice, e viceversa; sugli switch di transito il traffico entrato da una porta esce dall'altra.
3. **Dizionario *mac_to_port***, in cui il controller registra, per ogni switch, la porta da cui ha visto arrivare ciascun host. Poiché la porta di uscita è sempre quella della slice, un pacchetto non viene mai inoltrato su entrambi i percorsi.

Per ogni *PacketIn* il controller procede così:

1. **Broadcast**. Se la destinazione è broadcast (ad esempio una richiesta ARP), la destinazione logica diventa l'host partner del mittente.
2. **Controllo di accesso**. Se la coppia mittente–destinazione non è ammessa dalla tabella di controllo di accesso, il controller installa una regola di drop (nessuna azione, priorità 5) e si ferma.
3. **Learning**. Il controller registra in *mac_to_port* la porta da cui è arrivato il mittente.

4. **Porta di uscita.** La porta di uscita è letta da *mac_to_port*, se la destinazione è già nota, oppure dalla tabella di inoltro.
5. **Regola e inoltro.** Il controller installa la regola di inoltro (priorità 10), con match sulla destinazione reale del pacchetto, e lo reinvia con un ***PacketOut***.

Nel seguito, sono riportate le funzioni implementate per la realizzazione della logica descritta

```
def get_association_table(self, dpid: int) -> dict:
    if dpid in EDGE_SWITCH:
        return EDGE_SWITCH_PORT_ASSOCIATIONS
    elif dpid in CENTER_SWITCH:
        return CENTER_SWITCH_PORT_ASSOCIATIONS
    return None

def is_allowed_path(self, src, dst):
    allowed_destinations = ALLOWED_PATH.get(src)
    if not allowed_destinations or dst not in allowed_destinations:
        return False
    return True

def update_mac_to_port_table(self, dpid, src, in_port):
    if not self.mac_to_port.get(dpid):
        self.mac_to_port.setdefault(dpid, {})
        self.logger.info(f'[SWITCH] Create record for switch {dpid}')

    if not self.mac_to_port[dpid].get(src):
        self.mac_to_port[dpid][src] = in_port
        self.logger.info(f'[SWITCH] Update switch {dpid}, set {src} input port to: {in_port}')

def get_output_port(self, dpid, dst, in_port):
    out_port = self.mac_to_port[dpid].get(dst)
    if not out_port:
        association_table = self.get_association_table(dpid)
        if not association_table:
            return None

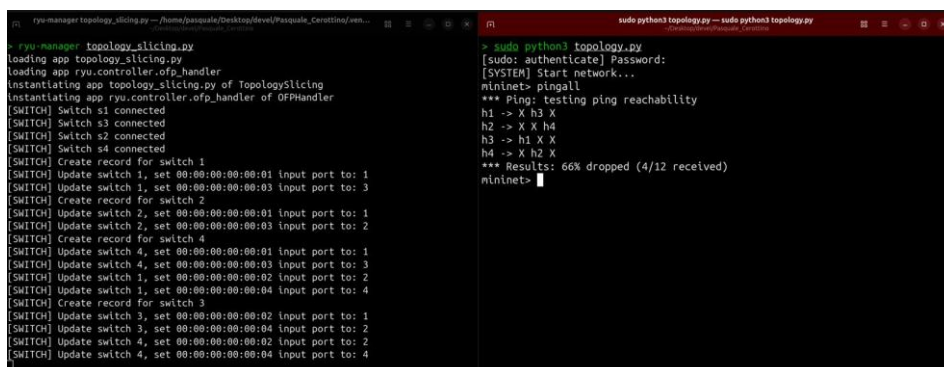
        out_port = association_table.get(in_port)
        if not out_port:
            return None

        self.update_mac_to_port_table(dpid, dst, out_port)

    return out_port
```

Verifica

I test eseguiti confermano l'isolamento previsto. Si riportano per alcuni gli strumenti usati e i risultati:



The image shows two terminal windows. The left window displays the output of the 'ryu-manager topology_slicing.py' command, showing the loading of the application and the instantiation of the OFPHandler. It lists several switches (s1, s2, s3, s4) and their connections, and shows the creation of records for each switch. The right window shows the output of the 'sudo python3 topology.py' command, which starts the network and performs a ping test. The ping test results show that 66% of the packets were dropped (4/12 received), indicating a network isolation issue.

```
ryu-manager topology_slicing.py --home /home/pasquale/Desktop/Deve/Pasquale_Cerottini/ven...
loading app topology_slicing.py
loading app ryu.controller.ofp_handler
instantiating app topology_slicing.py of TopologySlicing
instantiating app ryu.controller.ofp_handler of OFPHandler
[SWITCH] Switch s1 connected
[SWITCH] Switch s3 connected
[SWITCH] Switch s2 connected
[SWITCH] Switch s4 connected
[SWITCH] Create record for switch 1
[SWITCH] Update switch 1, set 00:00:00:00:00:01 input port to: 1
[SWITCH] Update switch 1, set 00:00:00:00:00:03 input port to: 3
[SWITCH] Create record for switch 2
[SWITCH] Update switch 2, set 00:00:00:00:00:01 input port to: 1
[SWITCH] Update switch 2, set 00:00:00:00:00:03 input port to: 2
[SWITCH] Create record for switch 4
[SWITCH] Update switch 4, set 00:00:00:00:00:01 input port to: 1
[SWITCH] Update switch 4, set 00:00:00:00:00:03 input port to: 3
[SWITCH] Update switch 1, set 00:00:00:00:00:02 input port to: 2
[SWITCH] Update switch 1, set 00:00:00:00:00:04 input port to: 4
[SWITCH] Create record for switch 3
[SWITCH] Update switch 3, set 00:00:00:00:00:02 input port to: 1
[SWITCH] Update switch 3, set 00:00:00:00:00:04 input port to: 2
[SWITCH] Update switch 4, set 00:00:00:00:00:02 input port to: 2
[SWITCH] Update switch 4, set 00:00:00:00:00:04 input port to: 4

> sudo python3 topology.py
[sudo: authenticate] Password:
[SYSTEM] Start network...
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X3 X
h2 -> X X4
h3 -> h1 X X
h4 -> X h2 X
*** Results: 66% dropped (4/12 received)
mininet>
```

Connettività (*pingall*). Risultato: **66% di pacchetti persi**. Rispondono solo le coppie H1–H3 e H2–H4.

```
mininet> h1 ping -c5 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=145 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=145 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=138 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=127 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=125 ms

--- 10.0.0.3 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 3996ms
rtt min/avg/max/mdev = 124.653/135.931/144.895/8.626 ms
mininet>

[sudo: authenticate] Password:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on s2-eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
19:32:12.171023 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 13259, seq 1, length 64
19:32:12.263126 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 13259, seq 1, length 64
19:32:13.258875 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 13259, seq 2, length 64
19:32:13.346090 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 13259, seq 2, length 64
19:32:14.332379 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 13259, seq 3, length 64
19:32:14.423586 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 13259, seq 3, length 64
19:32:15.416016 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 13259, seq 4, length 64
19:32:15.494697 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 13259, seq 4, length 64
19:32:16.496790 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 13259, seq 5, length 64
19:32:16.572384 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 13259, seq 5, length 64
```

Percorso della slice superiore (*tcpdump* su s2-eth1, link S1–S2). Risultato: solo traffico tra 10.0.0.1 e 10.0.0.3, catturato con l'esecuzione del comando *h1 ping – c5 h3*.

La lista completa dei test effettuati è disponibile del *Readme.md* del progetto.

4. Service Slicing

Il service slicing assegna la slice in base al tipo di traffico. Il traffico video, cioè IPv4 + UDP con porta di destinazione 9999, deve passare dalla slice superiore (S1–S2–S4, 10 *Mbit/s*); tutto il resto (ARP, ICMP, TCP, altro UDP) passa dalla slice inferiore (S1–S3–S4, 1 *Mbit/s*). A differenza del topology slicing, tutti gli host devono poter comunicare tra loro.

Progettazione

Il controller, implementato in *service_slicing.py*, è separato dal primo perché le due policy sono incompatibili: la prima deve bloccare la comunicazione tra H1 e H4, e tra H2 e H3 la seconda deve permetterla. Le scelte progettuali effettuate si basano su quelle illustrate nella sezione precedente:

1. Il controller **è reattivo**. Il primo pacchetto di ogni flusso risale al controller con un *PacketIn*: il controller lo classifica, installa le regole corrispondenti e lo reinvia con un *PacketOut*. Da quel momento i pacchetti dello stesso flusso vengono gestiti direttamente dallo switch.
2. **Le regole sono per flusso**. Ciascuna fa match sulla porta di ingresso e sulla coppia di MAC sorgente e destinazione; la regola video aggiunge i campi del protocollo e della porta UDP.

Alla connessione di uno switch il controller installa le stesse due regole del topology slicing: la table-miss (priorità 0) e lo scarto del traffico IPv6 (priorità 5).

Il controller distingue due classi di traffico:

1. **Video**: pacchetti IPv4 con protocollo 17 (UDP) e porta UDP di destinazione 9999. Seguono la slice superiore, le regole di questa classe hanno **priorità 20**.
2. **Non video**: tutto il resto (ARP, ICMP, TCP, altro UDP). Segue la slice inferiore, le regole di questa classe hanno **priorità 10**.

La logica del controller si basa su tre strutture dati:

1. **Mappa degli switch di bordo:** per S1 e S4 indica la porta di ciascun host locale e l'uplink di ciascuna slice. La classe del traffico conta solo qui, quando la destinazione si trova dall'altra parte della rete.
2. **Tabella di transito:** su S2 e S3 il traffico entrato da una porta esce dall'altra, indipendentemente dalla classe.
3. **Dizionario *mac_to_port*,** in cui il controller registra la porta degli host locali di ogni switch di bordo. La posizione degli host remoti non viene memorizzata, perché la porta per raggiungerli dipende dalla classe del traffico.

Per ogni *PacketIn* il controller procede così:

1. **Classificazione.** Il controller analizza le intestazioni del pacchetto e ne stabilisce la classe.
2. **Switch di transito.** Su S2 e S3 installa una regola sulla porta di ingresso verso l'altra porta e inoltra il pacchetto.
3. **Broadcast.** Su uno switch di bordo, un broadcast proveniente da un host viene inoltrato agli altri host locali e all'uplink della slice inferiore; uno proveniente da quell'uplink viene inoltrato a tutti gli host locali. La replica avviene con più azioni di output nella stessa regola, senza flooding, e il broadcast non attraversa mai la slice superiore, quindi non si creano cicli.
4. **Destinazione locale.** Se la destinazione è un host collegato allo stesso switch, il controller installa una sola regola verso la sua porta, valida per entrambe le classi.
5. **Destinazione remota.** Altrimenti installa due regole per la coppia di host, una generica (priorità 10, slice inferiore) e una video (priorità 20, slice superiore), e reinvia il pacchetto sull'uplink della sua classe.

Nel seguito è riportato di alcune delle funzioni che implementano la logica descritta:

```
def is_video_flow(self, ip_pkt, udp_pkt):
    return (
        ip_pkt is not None and ip_pkt.proto == in_proto.IPPROTO_UDP
        and udp_pkt is not None and udp_pkt.dst_port == VIDEO_UDP_PORT
    )

def update_mac_to_port_table(self, dpid, src, in_port):
    table = self.mac_to_port.setdefault(dpid, {})
    if table.get(src) != in_port:
        table[src] = in_port
        self.logger.info(f'[SWITCH] s({dpid}): learned {src} on port {in_port}')

def get_output_port(self, dpid, cfg, dst, in_port, is_video):
    local_port = self.mac_to_port.get(dpid, {}).get(dst)
    if local_port:
        return local_port

    if in_port not in cfg['host_ports']:
        return None

    return cfg['video'] if is_video else cfg['other']
```

```
def install_remote_rules(self, datapath, cfg, src, dst):
    parser = datapath.ofproto_parser

    self.add_flow(
        datapath, FORWARD_PRIORITY,
        parser.OFPMatch(eth_src=src, eth_dst=dst),
        [parser.OFPACTIONOutput(cfg['other'])])

    video_match = parser.OFPMatch(
        eth_src=src, eth_dst=dst,
        eth_type=ether_types.ETH_TYPE_IP,
        ip_proto=in_proto.IPPROTO_UDP,
        udp_dst=VIDEO_UDP_PORT
    )

    self.add_flow(
        datapath, VIDEO_PRIORITY, video_match, [parser.OFPACTIONOutput(cfg['video'])])

    self.logger.info(
        f'[SWITCH] s({datapath.id}): rules {src} -> {dst} (video -> {cfg["video"]}, other -> {cfg["other"]})')
```

Verifica

I test eseguiti confermano l'isolamento previsto. Si riportano per alcuni gli strumenti usati e i risultati:

```
* ryu-manager topology_service.py
loading app topology_service.py
loading app ryu.controller.ofp_handler
instantiating app topology_service.py of Serviceslicing
instantiating app ryu.controller.ofp_handler of OFPHandler
[SWITCH] Switch s1 connected
[SWITCH] Switch s4 connected
[SWITCH] Switch s2 connected
[SWITCH] Switch s3 connected
[SWITCH] s1: learned 00:00:00:00:00:01 on port 1
[SWITCH] s1: learned 00:00:00:00:00:02 on port 2
[SWITCH] s4: learned 00:00:00:00:00:03 on port 3
[SWITCH] s4: rules 00:00:00:00:00:03 -> 00:00:00:00:00:01 (video -> 1, other -> 2)
[SWITCH] s1: rules 00:00:00:00:00:01 -> 00:00:00:00:00:03 (video -> 3, other -> 4)
[SWITCH] s4: learned 00:00:00:00:00:04 on port 4
[SWITCH] s4: rules 00:00:00:00:00:04 -> 00:00:00:00:00:01 (video -> 1, other -> 2)
[SWITCH] s1: rules 00:00:00:00:00:01 -> 00:00:00:00:00:04 (video -> 3, other -> 4)
[SWITCH] s4: rules 00:00:00:00:00:03 -> 00:00:00:00:00:02 (video -> 1, other -> 2)
[SWITCH] s1: rules 00:00:00:00:00:02 -> 00:00:00:00:00:03 (video -> 3, other -> 4)
[SWITCH] s4: rules 00:00:00:00:00:04 -> 00:00:00:00:00:02 (video -> 1, other -> 2)
[SWITCH] s1: rules 00:00:00:00:00:02 -> 00:00:00:00:00:04 (video -> 3, other -> 4)

> sudo python3 topology.py
[sudo: authenticate] Password:
[SYSTEM] Start network...
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet>
```

Connettività (*pingall*). Risultato atteso: **0% di pacchetti persi**, il risultato è dato dal fatto che tutti gli host comunicano tra loro.

```
mininet> h3 iperf -s -u -p 9999 &
mininet> h1 iperf -c 10.0.0.3 -u -p 9999 -b 10M -t 5
-----
Client connecting to 10.0.0.3, UDP port 9999
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.1 port 49354 connected with 10.0.0.3 port 9999
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-5.0014 sec 6.25 MBytes 10.5 Mbits/sec
[ 1] Sent 4461 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-5.2718 sec 6.11 MBytes 9.73 Mbits/sec 0.000 ms 99/4460 (0%)
[ 1] 0.0000-5.2718 sec 100 datagrams received out-of-order
mininet>
```

Banda del traffico video (*iperf* UDP da H1 a H3, porta 9999, 10 Mbit/s per 5 s). Risultato: **9,73 Mbit/s ricevuti da H3**, praticamente il limite del link a 10 Mbit/s; il flusso è quindi passato dalla slice superiore.

La lista completa dei test effettuati è disponibile del *Readme.md* del progetto.